

COMP 3625 Assignment 1

Search and optimization

This assignment will become visible in D2L *after* you join a D2L group.

You **must** work in a group of 3~4 for this assignment. Individual assignments by permission only.

Overview

The problem

You are the engineer on a starship that has just encountered a new alien ship. The alien ship transmits a message to your captain, but the ship's universal translator is broken - you cannot translate the message.

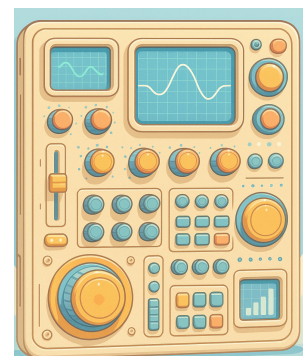
You must write a program to help you get the translator back online. Hurry! If the alien's intentions are hostile, there's no time to lose!



The task

You receive the alien message as a string of numeric codes. The Universal Translator is capable of decoding each number into an English word, but it has several knobs that must be set to tune its performance. Someone has been fiddling with these knobs, and the translator is out-of-whack.

Some knob settings will decode only a few of the words, while others will decode more. Decoding 100% may not be possible.



Note the translator performance varies *smoothly*: a small change in settings will not cause large, discontinuous jumps in performance.

Write a program to search for the ideal knob settings. Your program will be evaluated based on two things:

1. What fraction of the words are successfully decoded (ranges from 0-1, more is better)
2. How many setting combinations were tried (less is better - remember your captain wants a solution ASAP!)

Starter code

You are provided with a `UniversalTranslator` class that represents the translator. When you instantiate a `UniversalTranslator` object, you must specify a number of knobs (the `n_dims` parameter).

You can then pass settings to the `translate` method to get the translator's attempted translation using those settings. Note that the settings passed to `translate` must be an **array or list of values** that are **each in the range [0, 1]**. There must be **one value for each knob**.

The object also has a `n_settings_tried` method which will return the total number of translation attempts made so far.

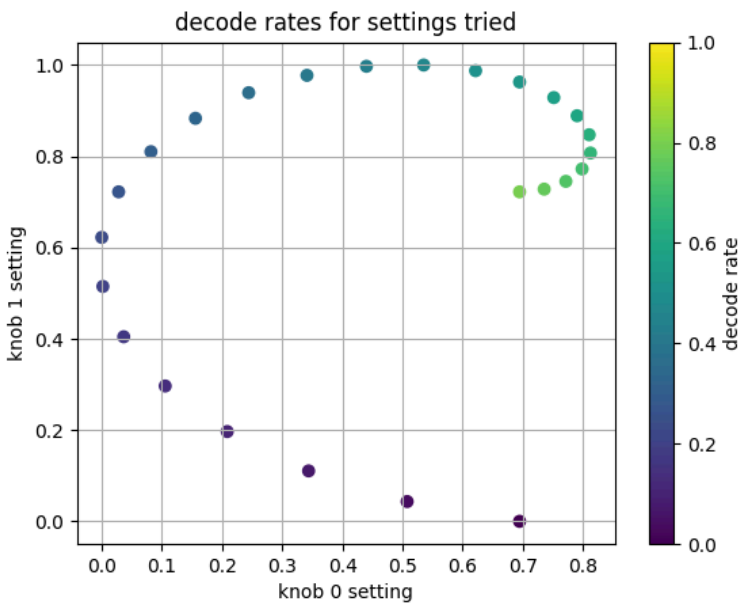
Assignment

Part 1

In `part1.py`, instantiate a `UniversalTranslator` object with **2 knobs**, and write a search program of some kind to find the ideal settings. You can write functions, classes, and `.py` files as necessary, but running `part1.py` must run the search and then clearly display:

1. The best settings found, and the decode-rate achieved with these settings
2. The total number of settings tried

You must also generate a 2D plot of all the settings tried, and the decode-rate associated with each. Plot the 2 setting values on the X and Y axes, use color to indicate the decode-rate. Something like this:



The following code snippet shows one way to make a plot like this, but you don't necessarily have to use it:

Python

```
# given settings: a Nx2 array of N setting combinations
# and decode_rate: a length-N array of decode rates corresponding to those
settings

# generate a scatter plot
plt.scatter(x=settings[:, 0],
            y=settings[:, 1],
            c=decode_rate,
            vmin=0, vmax=1
            )

# add colorbar and gridlines
cbar = plt.colorbar(label="decode rate")
plt.grid()

# add labels
plt.xlabel('knob 0 setting')
plt.ylabel('knob 1 setting')
plt.title('decode rates for settings tried')
```

```
# display  
plt.show()
```

Part 2

In `part2.py`, repeat the search problem for a `UniversalTranslator` object with **10 knobs**. You **may or may not choose** to change up your search strategy from part 1. You can add new code as necessary, or make calls out to the functions you wrote for part 1, as long as running `part2.py` runs the 10-knob search and then clearly displays:

1. The best settings found, and the decode-rate achieved with these settings
2. The total number of settings tried

No plot is necessary for this part, since it's difficult to visualize things in 10 dimensions :)

Written responses

After you finish coding, tackle the questions in `written_responses.md`

Demos and friendly competition

On an assigned day, you will briefly explain your program to the class, explaining and justifying your design decisions.

I will then test your program on a 10-knob problem (a new one with a different solution), and measure (1) the final decode rate, and (2) number of settings tried. Teams will be compared on both these dimensions, and teams falling **on the Pareto Front** will win bragging rights.

Requirements

- Since I need to run all the 10-knob searches on my side, your 10-knob search code must run in **60 seconds or less**.

- Your program must not return a random or trivial solution.
- Don't modify translator.py. Modifying, decompiling or otherwise reverse-engineering translator_core_ob is cheating (and wouldn't help you anyway!)
- In general, stick to search methods discussed in class, or variations/combinations of those.
If you want to try something different, great! Just clear it with Eric first.

Evaluation

Per the MRU grading table in the course outline, an “A” solution represents “Excellent performance, showing comprehensive understanding of subject matter.”

An “A” submission for this assignment will:

- Perform well & meet requirements
- Be well structured, easy to read and understand
- Show significant understanding of search problems and techniques
- Have clear & concise written responses that clearly demonstrate understanding
- Include a clear demonstration that explains what the program is doing and why you built it that way
- Include git commit history that shows a progression of work contributed by all group members.

A “C” submission represents “Basic understanding of subject matter”, and might:

- Include sloppy, difficult-to-read code. Possibly one big dump of code into github on the due date.
- Not perform much better than a trivial or brute-force solution
- Include written responses that are vague, surface-level, off-topic, or rambling.
- Include a demo in which choices are not justified or a lack of effort is apparent.
 - If it becomes clear during the demo that you submitted code you do not understand, you will most likely receive a **failing grade**!

Submission

Enter a **github repository url** into the D2L assignment.

Make sure you add me (echalmers) as a collaborator on the repo, so I have read access.

Credits:

Alien script and spaceship/control panel pictures created by ChatGPT